Release Gates Should Ask Where the Carbon Number Came From

Adhithya Rajasekaran, Dhinakaran Thangaraj, Lakshman S, Navya Velicheti
Axonome
adhi.r@axonome.xyz, www.dhina555@gmail.com, slakshman2004@gmail.com, navyavelicheti28@gmail.com

***Abstract*—Green software practice increasingly puts a carbon number in front of a release decision: a carbon budget, a readiness level, or a risk rating. None of these gates asks whether the number itself can be trusted. We argue that the *provenance* of an environmental claim should be a first-class release criterion, and we report on FairMind-E, an environmental release gate shipped inside FairMind, an open-source AI governance platform. A provenance-derived confidence score crossed with an impact tier resolves to *go*, *conditional_go*, or *no_go*, and blocks the platform's deployment-approval workflow. In a measurement study over four workloads, five iterations each, a software power tracker and an analytic FLOP model assessed every run and disagreed on magnitude by 123–169 percent MAPE, depending on which is taken as reference. One routine mitigation failed outright: casting a fine-tune to `bfloat16` on a host with no optimised kernel made it 46 times slower, and the tracker estimated roughly 41 times the energy. A gate that accepts declared mitigations would have logged a reduction that never happened.**

***Index Terms*—green software, sustainable AI, release engineering, carbon measurement, evidence quality, AI governance**

I. Position

Software teams are being asked to account for the energy and carbon of the AI systems they ship. Standards bodies are building the measurement layer: quantification-methods inventories [1], sustainability overviews [2], and a G7-commissioned inventory of thirteen measurement and reporting initiatives [3]. The EU AI Act anticipates harmonised standards for reporting AI resource performance (Article 40) and requires general-purpose model documentation to state known or estimated energy consumption (Annex XI) [4]. None of these instruments says how the *quality* of an environmental claim should condition a deployment decision.

Proposed carbon-aware gates decide on magnitude. Carbon-aware governance gates embed carbon budgets and record provenance only in audit logs [5]; sustainability pyramids gate on readiness levels [6]; risk integrations treat AI carbon as one risk dimension [7]. Yet measurement research shows reliability varies sharply with disclosure: metadata-completeness tiers across 5,227 public models span roughly $1.2\times$ to $2\times$ estimation error [8], and spend-based fallbacks in tiered Scope 3 accounting overestimate by 10–40$\times$ [9]. Recent work acts on the *statistical spread* of an estimate, through conformal calibration inside a carbon budget gate [10] or confidence-gated filtering of emissions predictions [11]. None acts on *where the number came from*, so a precise-looking figure from a weak source passes every published gate while sitting comfortably under budget.

Our position is narrow: **for green software engineering, evidence quality, not only impact magnitude, should gate a release.** The idea is old in life-cycle assessment, where pedigree matrices grade the provenance of an inventory figure [12] and the NUSAP pedigree qualifies uncertainty [13]; recent work recommends indicator-style disclosure for AI-assisted footprinting but proposes no score, threshold, or gate [14]. We bring it into the release pipeline, where software engineers already make go or no-go decisions.

II. Mechanism

a) Provenance classes and confidence: Every assessment declares its measurement source, normalised to one of five provenance classes with bounded confidence: *measured* (hardware telemetry; 0.85–1.00), *tool_estimated* (software power trackers; 0.60–0.85), *vendor_reported* (cloud-provider figures; 0.40–0.60), *manual* (analytic FLOP estimates; 0.20–0.40), and *unknown* (0.0). A caller-supplied score is honoured but capped by its provenance class, and unknown provenance forces 0.0 whatever score is claimed. Provenance is reported separately from quantified uncertainty, so a narrow interval cannot launder a weak source.

b) Tiers and the decision matrix: Metrics map to low, medium, or high impact per lifecycle phase, using location-based carbon so market instruments cannot lower a gate outcome. Confidence bands (*high* ≥ 0.70; *moderate* $0.35 \leq c < 0.70$; *low* $0 < c < 0.35$; *undisclosed* $c = 0$) cross with impact tiers into the matrix in Fig. 1(b). Confidence 0.0 short-circuits to *no_go*: an undisclosed claim never passes. A *conditional_go* cannot be approved until it carries a dated, owned mitigation or an explicit exception with owner, expiry, and rationale, so reviewer overrides are auditable rather than informal. Six controls (ENV-1 to ENV-6: boundary, energy accounting, dual location and market carbon, per-functional-unit efficiency, evidence confidence with disclosed uncertainty, and mitigation or exception) define coverage. Disclosed offsets are recorded but never subtracted and never improve a gate outcome.

III. Implementation in a release pipeline

FairMind-E ships inside FairMind rather than as a standalone proposal. The decision engine is a framework-agnostic Python package with 57 unit tests covering every matrix cell, band boundary, provenance cap, and the undisclosed-evidence short-circuit. Assessments persist as versioned governance

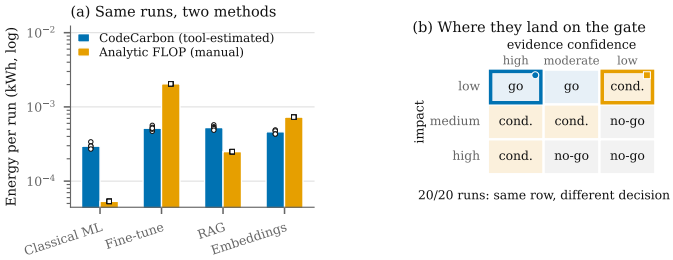


Fig. 1. Four workloads, five iterations each, every run assessed twice. (a) The methods disagree by 123–169 percent MAPE depending on reference; points are iterations, bars their means. (b) Where those runs land. Both put every run in the same *low* row, so the column moved because provenance moved. The panel illustrates the mechanism; it does not validate it.

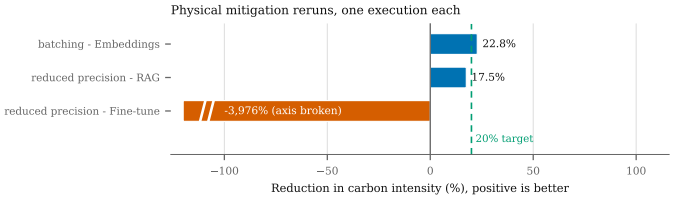


Fig. 2. Mitigation outcomes by what was executed. Hatched bars are accounting scenarios recomputed against a *placeholder* emission-factor table with no rerun: illustrative arithmetic, not measurements. Solid bars are the three physical reruns. The `bfloat16` fine-tune went from 62.7 s to 2887.6 s; its bar is truncated.

evidence. A release gate hooks the platform's deployment-approval workflow and returns HTTP 409 with the blocking reason when the latest assessment is *no_go* or an unmitigated *conditional_go*, the same way a failing test blocks a merge. Reviewers see the matrix position, control coverage, mitigations, and version trail; disclosures export in a shape aligned with the European climate reporting standard [15].

A 14-case hand-labelled fixture set spanning all matrix cells, mitigation states, exception paths, and offset disclosures serves as a conformance test: the implementation reproduces 14/14 labels. Because the same authors wrote the fixtures, labels, and engine, this shows only that the code implements the stated policy, not that the policy decides correctly.

IV. Measurement Study

Four workloads (classical ML training, transformer fine-tune, RAG inference, batch embeddings) ran five iterations each on one host (Apple M1 Pro; seed 1337; warm-up discarded). Every run was assessed by a software power tracker (*tool_estimated*) and a documented analytic FLOP model (*manual*); cloud billing was also requested but lags execution, so its 20 records are explicit "unavailable" entries rather than invented figures. All runs were replayed through the production engine, not a reimplementation.

a) The empirical finding is the disagreement: MAPE is 122.6 percent with the tracker as reference and 169.1 percent with the FLOP model as reference, ranging from 52 to 298 percent by workload, with Bland-Altman limits of agreement from -1.15×10^{-3} to 1.79×10^{-3} kWh (Fig. 1(a)). Two methods a practitioner might reasonably choose disagree by more than a factor of two on the same run.

b) The decision split is a policy consequence: The tracker maps to confidence 0.72 (*high* band, *go*); the FLOP model to 0.30 (*low* band, *conditional_go*), on all 20 runs. Since confidence is constant per provenance class and the tier stayed *low*, the split follows from the policy and is not independent evidence for it. What it does show is that the mechanism runs end to end on real runs, and that a magnitude-only gate is structurally blind to the split.

c) Mitigations: Only three mitigations were physical reruns (Fig. 2); region and time shifts recompute one measured run against placeholder factors and are reported as arithmetic, not reductions. Batching cut intensity 22.8 percent and a reduced-precision cast on RAG 17.5 percent. The third inverted: casting the fine-tune to `bfloat16` made it 46 times slower on a host with no optimised kernel, and the tracker estimated roughly 41 times the energy. The runtime regression is measured directly; the energy figure inherits the tracker's power model. "We reduced precision, therefore we cut carbon" is a routine and plausible claim; here it was false in the harmful direction. This is why ENV-6 requires a mitigation to be re-measured rather than declared.

V. Implications for Green Software Engineering

a) Make measurement the cheapest path to release: When evidence quality gates a release, a team that cannot substantiate its number cannot ship unconditionally, so measuring becomes cheaper than arguing. This does not cut emissions by itself; it makes reported reductions verifiable, and therefore something a team can act on.

b) Treat mitigations like code changes: A mitigation is a claim about a change. Like a performance fix, it should be re-measured in the pipeline before it is credited, because the effect of a precision cast or a batching change depends on the hardware and kernels underneath.

c) Open questions for the workshop: Is the provenance hierarchy the right one, and how should it be calibrated against an independent power reference? Which confidence bands should a CI pipeline use when only software estimates exist, as on most developer hardware? And how should overrides be audited so that *conditional_go* does not become the new default?

d) Limitations: The host exposes no hardware energy counters, so the tracker fell back to load-based estimation: the study compares an estimate with an estimate and never claims the *measured* class. Benchmark workloads exercise only the low-impact row, the FLOP model uses placeholder constants, and confidence is not yet calibrated against an independent power reference; that calibration, and a 15-reviewer expert panel already designed, are the next steps. Code, run data, and analysis scripts are public in the FairMind repository (https://github.com/adhit-r/fairmind, release v0.3.0-alpha, doi:10.5281/zenodo.24043235).

Acknowledgment

FairMind is developed at Axonome, where the authors work; the work received no external funding. Generative AI tools (Claude, through Claude Code) assisted with code and drafting; the authors reviewed all output and are responsible for the content.

References

[1] CEN/CLC/TR 18145:2025, *Environmentally sustainable artificial intelligence: quantification methods inventory*, 2025.
[2] ISO/IEC TR 20226:2025, *Artificial intelligence: environmental sustainability aspects*, 2025.
[3] G7 French Presidency, *Overview of initiatives for the measurement and monitoring of energy and resource use of AI models*, May 2026.
[4] Regulation (EU) 2024/1689 (Artificial Intelligence Act), Article 40 and Annex XI §1(2d), 2024.
[5] "Carbon-aware governance gates: embedding carbon budgets, energy provenance, and sustainability-aware validation in governance layers," arXiv:2602.19718v2, 2026.
[6] "G-TRACE: an AI sustainability pyramid for carbon-aware governance readiness," arXiv:2511.04776v3, 2026.
[7] N. Tkachenko, "Sustainable integration of frontier AI in banking," *Journal of Financial Compliance*, 2025.
[8] "Hugging Carbon: estimation reliability tiers for the carbon footprint of 5,227 public models," arXiv:2605.01549v2, 2026.
[9] "A four-tier Scope 3 methodology for AI inference under CSRD," arXiv:2606.10660, 2026.
[10] "Green SARC: predictive cost and carbon governance for agentic AI with split-conformal calibration," arXiv:2606.15954, 2026.
[11] "Trust-aware predictive emissions monitoring: confidence-gated filtering of industrial emissions estimates," arXiv:2606.06156, 2026.
[12] B. P. Weidema and M. S. Wesnæs, "Data quality management for life cycle inventories: an example of using data quality indicators," *Journal of Cleaner Production*, vol. 4, no. 3–4, pp. 167–174, 1996.
[13] S. O. Funtowicz and J. R. Ravetz, *Uncertainty and Quality in Science for Policy*. Dordrecht: Kluwer, 1990.
[14] "Criteria for credible AI-assisted carbon footprinting systems," arXiv:2509.00240, 2025.
[15] Directive (EU) 2022/2464 (Corporate Sustainability Reporting Directive) and ESRS E1, Climate change, 2023.